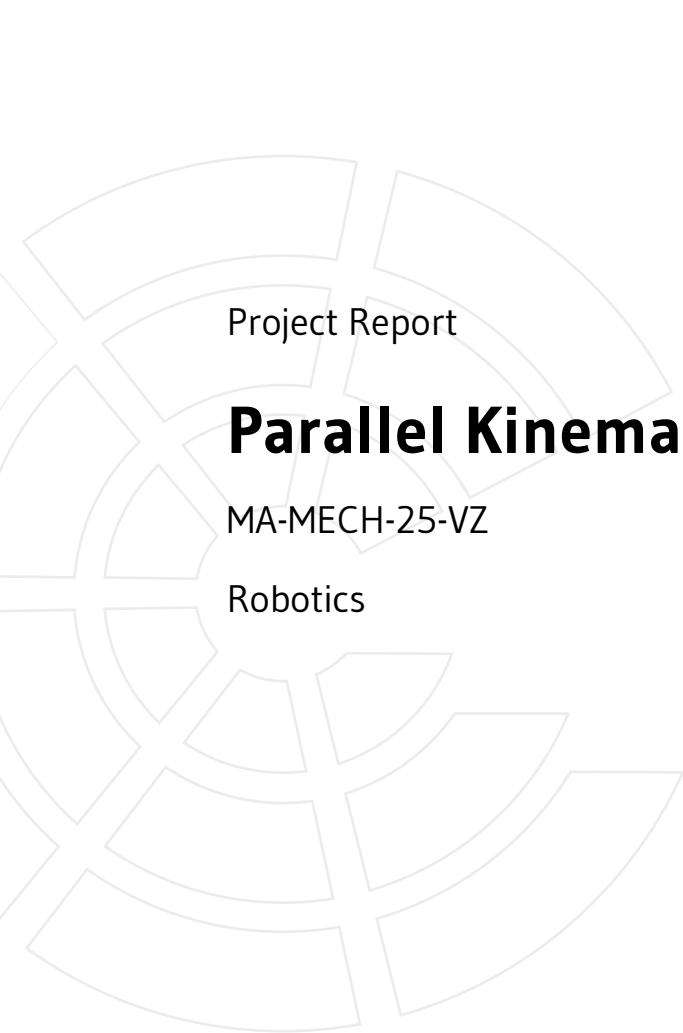




Project Report

Parallel Kinematics of a Delta Robot

MA-MECH-25-VZ

Robotics

Author(s)

Student ID(s)

Cohort

Lecturer

Andreas Thöni

2510620039

MA-MECH-25-VZ

Mehmet Ismet Can Dede, PhD

Contents

1	Task description	1
2	Simscape Multibody Modeling	2
3	Forward and Inverse Kinematics	3
3.1	Forward Kinematics	3
3.2	Inverse Kinematics	4
4	Joint Space Motion	6
5	Task Space Motion	8
5.1	Trapezoidal trajectory in task space	8
5.2	Comparison joint and task space	9
6	Continuous Trajectory	11

Chapter 1

Task description

The task of this project was to generate a model for the Delta 360 parallel manipulator by IGUS in MATLAB using the Simscape Multibody blockset. After generating the model, several MATLAB functions should be implemented to compare theoretical calculations of kinematics with the simulation, move in joint- and task-space, and create suitable trajectories.

Chapter 2

Simscape Multibody Modeling

The Multibody model of the robot has been created using the Simscape Multibody Link Plugin and several files that have been given by the lecturer. Using the plugin, it is possible to create a simulation from an exported .xml file and several .step-files, both exported from CAD-software, in this case from Solidworks. The resulting model has then been slightly adjusted to be usable for the tasks at hand. These adjustments include:

- the addition of a transform sensor, which is able to measure the relative transform between the origin of our chosen coordinate system and the end-effector.
- the addition of several transform adjustments, to ensure the correct detection of the relative transform.
- the addition of position input ports to the prismatic joints, to be able to control them accordingly.
- the addition of three subsystems, which automatically compute the first two derivatives of the input position information and perform a sign-correction.

A screenshot of how the adjusted model looks can be seen in figure 2.1.

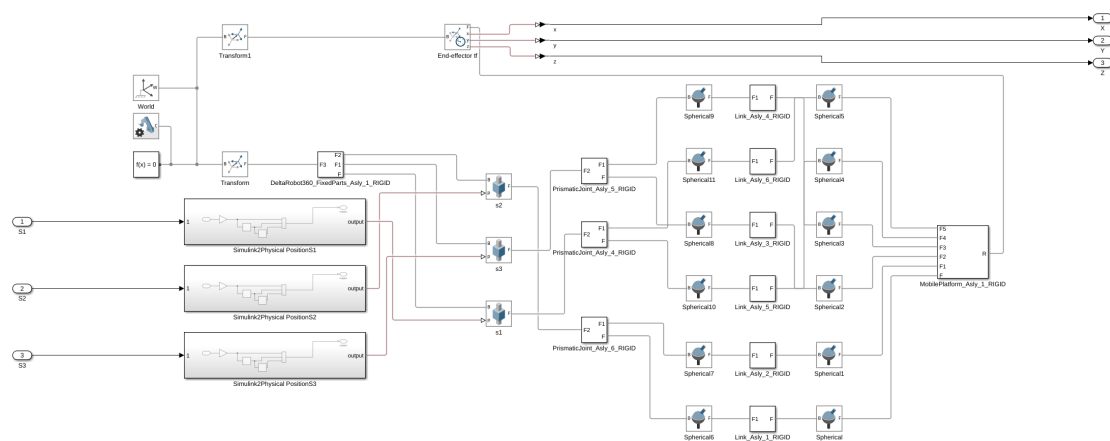


Figure 2.1: Adjusted Simscape Multibody model of the robot

Chapter 3

Forward and Inverse Kinematics

3.1 Forward Kinematics

Calculating forward kinematics is the process of determining the endeffector position depending on the joint variables. The equations for calculating the forward kinematics as well as the needed dimensions of the Delta robot have been taken from [1] and were put inside a MATLAB-function block to be usable inside Simulink. The code of the function block can be found in *forward_kinematics.m*.

To be able to verify if the equations have been implemented correctly, the euclidean error between the Multibody simulation (MB) and the output of the forward kinematics function (FK) has been calculated. The results can be seen in figure 3.1.

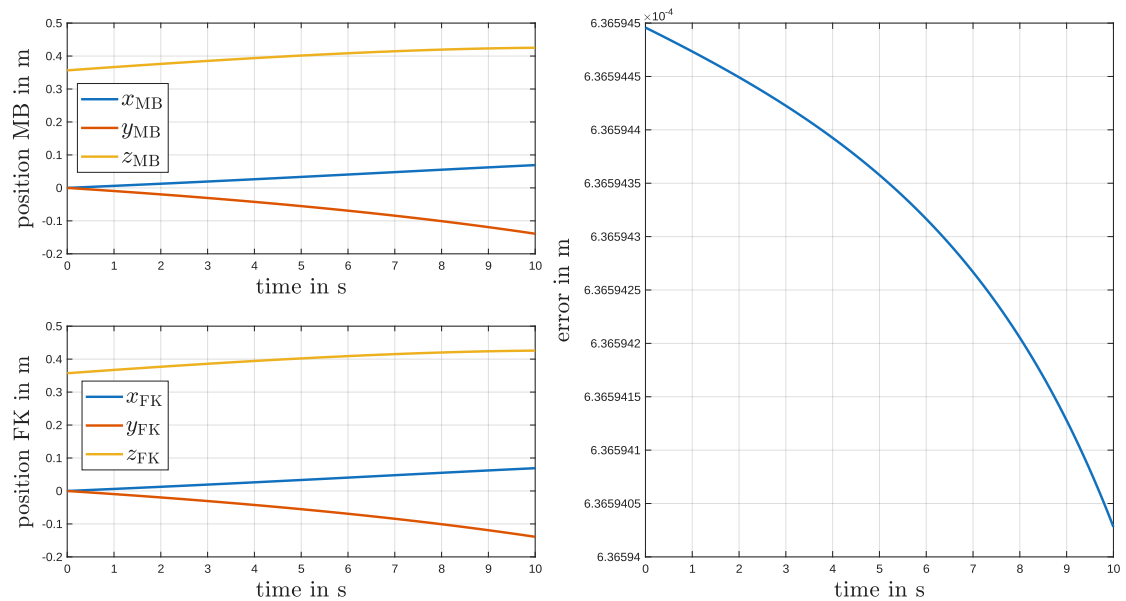


Figure 3.1: Plot of Verification process for forward kinematics. Left column: position calculated by Multibody model (MB) and function (FK); right column: euclidean error of the two

The euclidean error is calculated as described in equation (3.1).

$$e = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2} \quad (3.1)$$

The left column in figure 3.1 shows that for the joint trajectory that has been given to the system, the position of the end-effector in simulation and the forward kinematics function yield the same result. The plot in the right column shows an error of magnitudes of 10^{-4} m.

3.2 Inverse Kinematics

Calculating inverse kinematics is the process of determining the joint positions necessary for the end-effector to be at a given position in task space. The equations for inverse kinematics for the robot are also taken from [1]. A function called *inverse_kinematics.m* has been implemented to do the conversion. To verify the results, the end-effector output of the Multibody-model has been fed into the inverse kinematics function, whose output has been compared to the input into the Simulation. Like above, the inputs are three ramps with different inclinations. The results can be seen in figure 3.2.

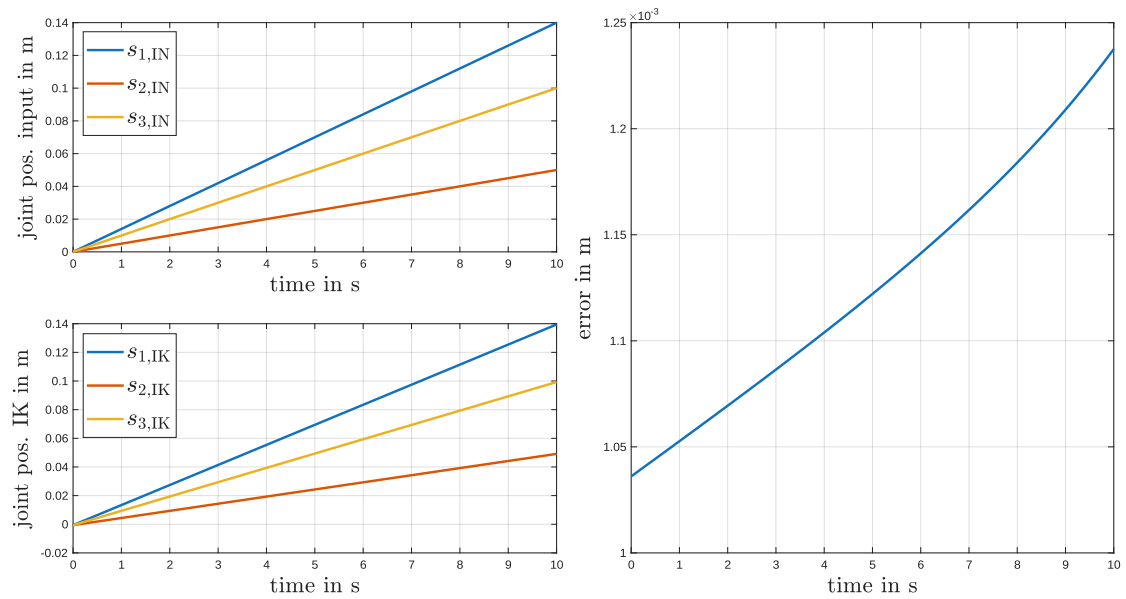


Figure 3.2: Plot of verification process for inverse kinematics. Left column: input joint position into Multibody model (IN) and output of inverse kinematics function (IK); right column: euclidean error of the two

It is apparent that the inverse kinematics (IK) function yields the same results as the joint positions that were input into the simulation. The error, however, has a magnitude of 10^{-3} m, which is larger than for forward kinematics.

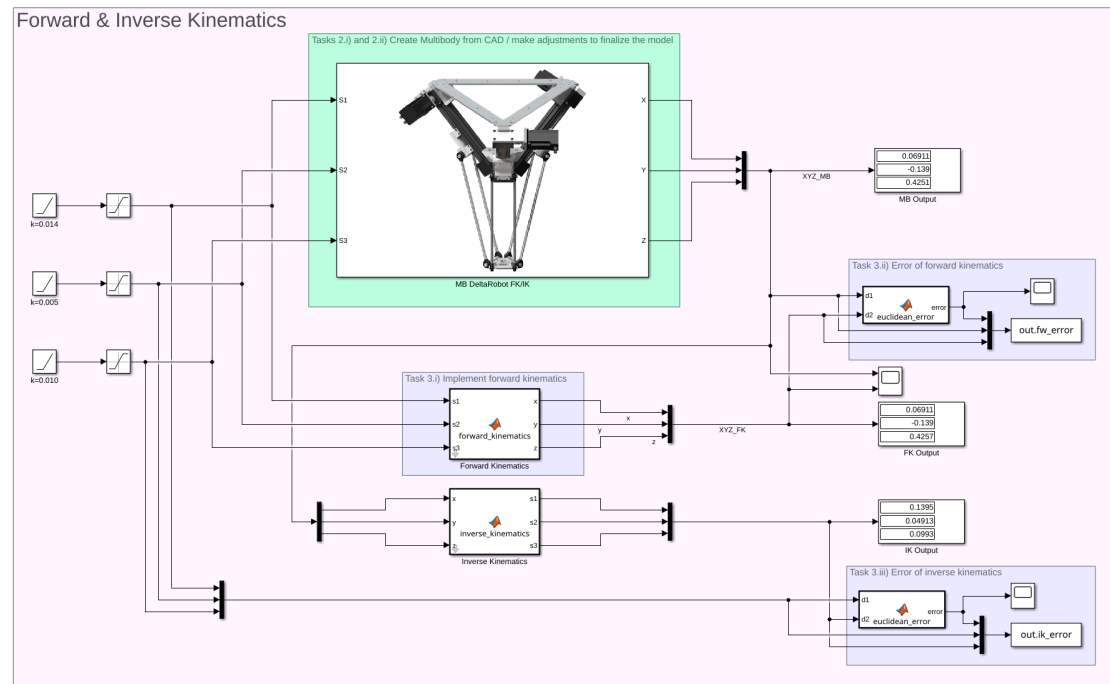


Figure 3.3: Simulink layout for verifying forward and inverse kinematics

A screenshot of the section of the Simulink model that has been used to fulfill these tasks can be seen in figure 3.3.

Chapter 4

Joint Space Motion

In this section, the process of driving joint space motions with the robot is documented. First, three waypoints are chosen inside of the workspace of the robot. They can be seen in table 4.1.

Table 4.1: Chosen waypoints for trajectory planning

Waypoint	x in m	y in m	z in m
1	0.0	0.1	0.4
2	0.075	-0.075	0.45
3	-0.05	-0.05	0.4

Using the acceleration and velocity constraints that were given in the task, the synchronized velocity profile has been calculated in the file *jspace_trapz.m*. The outputs are time stamps and corresponding velocity values, which are then input into a Repeating table block in Simulink and fed into an integrator to receive the needed position information. As the simulation uses the vectors from the workspace, the script *jspace_trapz.m* needs to be run before the simulation.

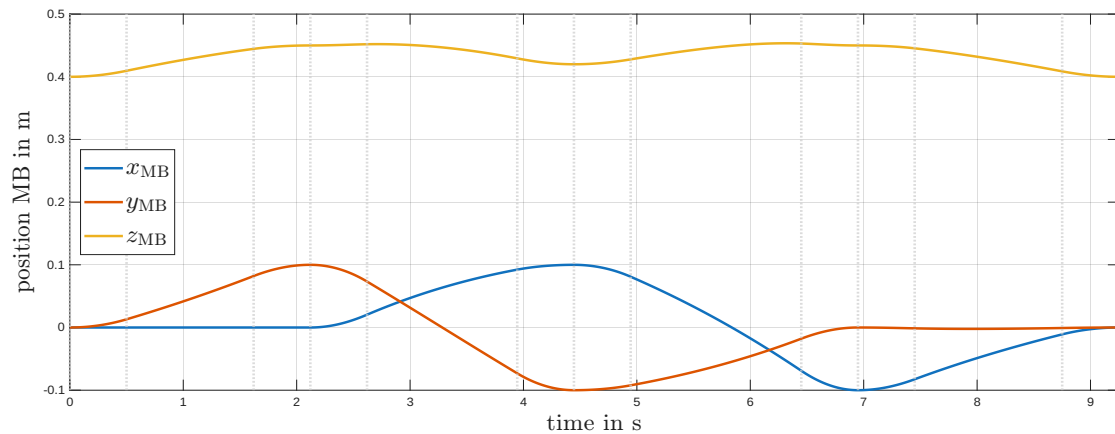


Figure 4.1: x , y and z position of the end-effector for the trapezoidal velocity profile in joint space

Figure 4.1 shows the position of the end-effector of the robot when following the trapezoidal trajectory. A more visually appealing plot is shown later in section 5.2. The trajectory itself, with position,

velocity and acceleration profile can be viewed in figure 4.2.

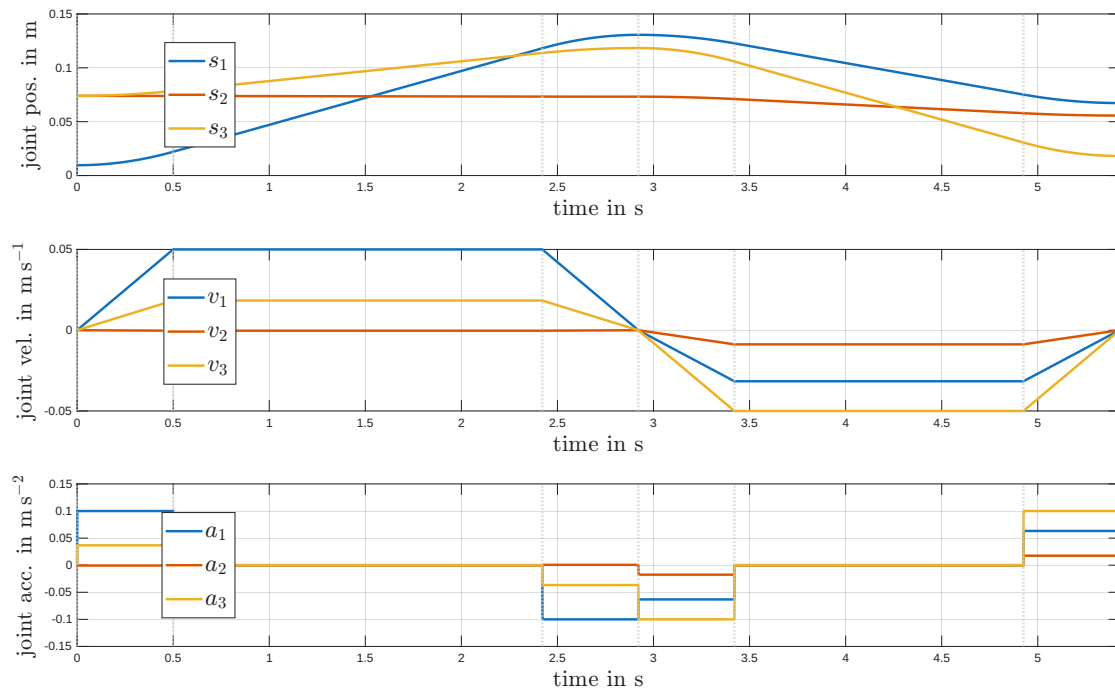


Figure 4.2: Generated trajectory, including position, velocity and acceleration profiles in joint space

The maximum velocity should not exceed 0.05 m s^{-1} , while the acceleration should be capped at 0.1 m s^{-2} . It is apparent, that none of the velocity or acceleration constraints are violated.

Chapter 5

Task Space Motion

This section document the implementation of movements in task space.

5.1 Trapezoidal trajectory in task space

The same waypoints from the joint space motion have been used, which are visible in table 4.1. The maximum velocity of the end-effector was set to be 0.04 m s^{-1} , the maximum acceleration 0.07 m s^{-2} . The motion of the end-effector can be viewed in figure 5.1.

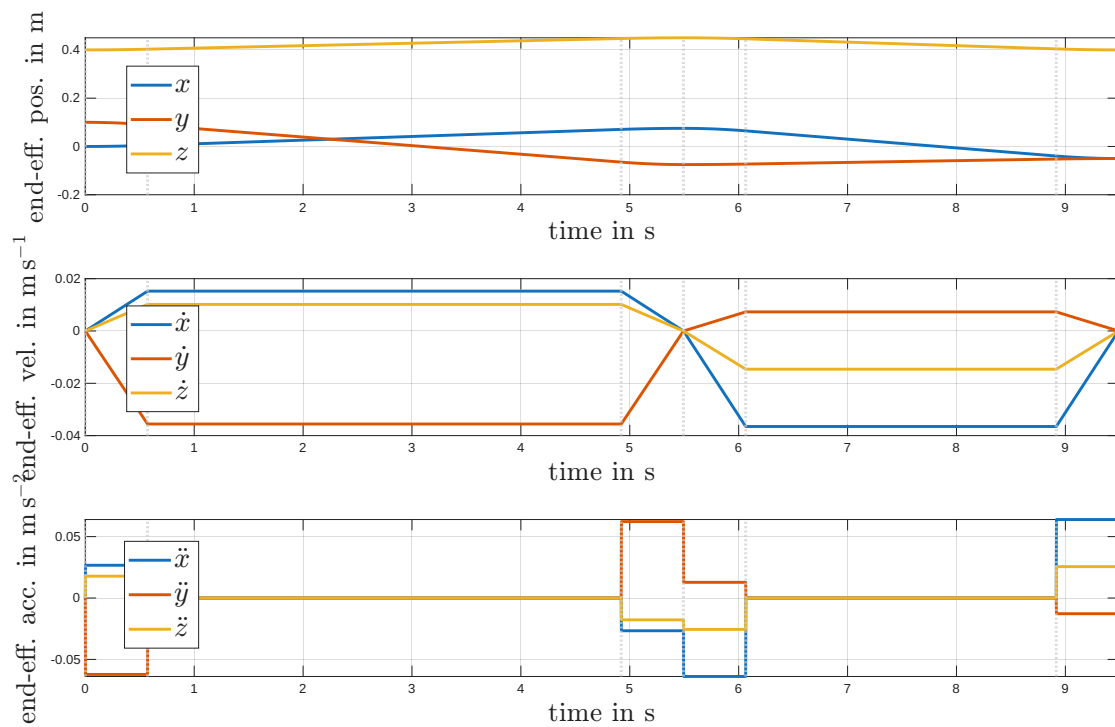


Figure 5.1: Position, velocity and acceleration of the end-effector for task space motion

As expected, the end-effector moves in a straight line from point to point. The acceleration and

velocity constraints are fulfilled.

5.2 Comparison joint and task space

However, compared to the joint space motion, the task space motion takes more time to execute within the constraints. A comparison of the joint velocities and accelerations for both can be viewed in figure 5.2.

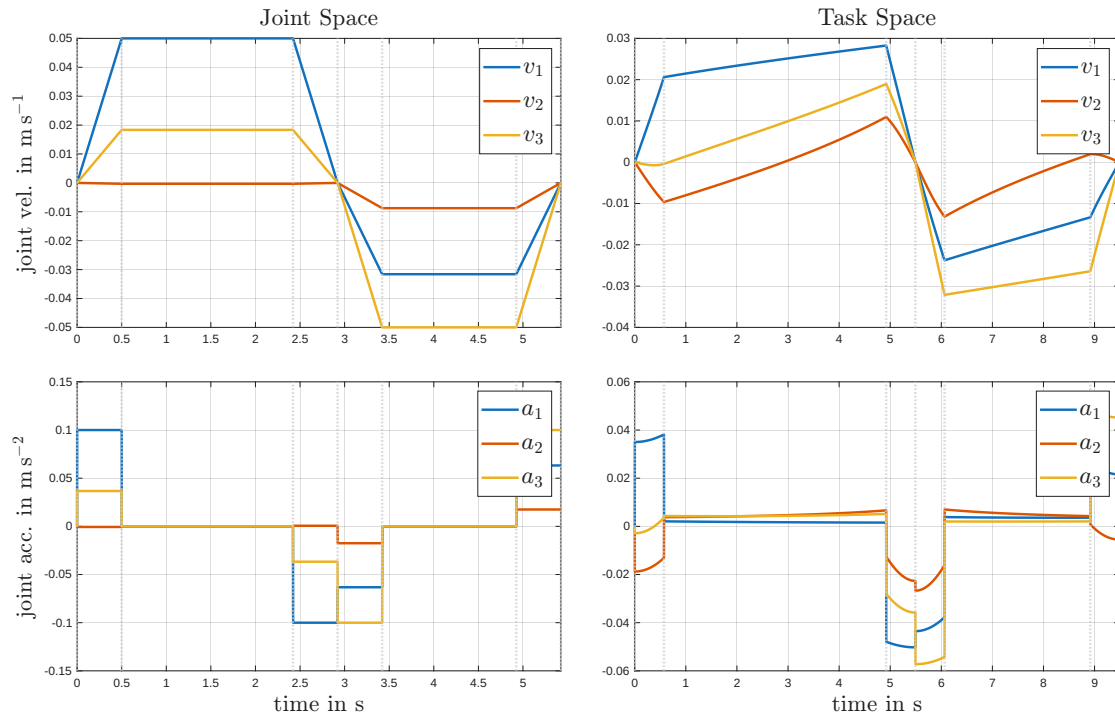
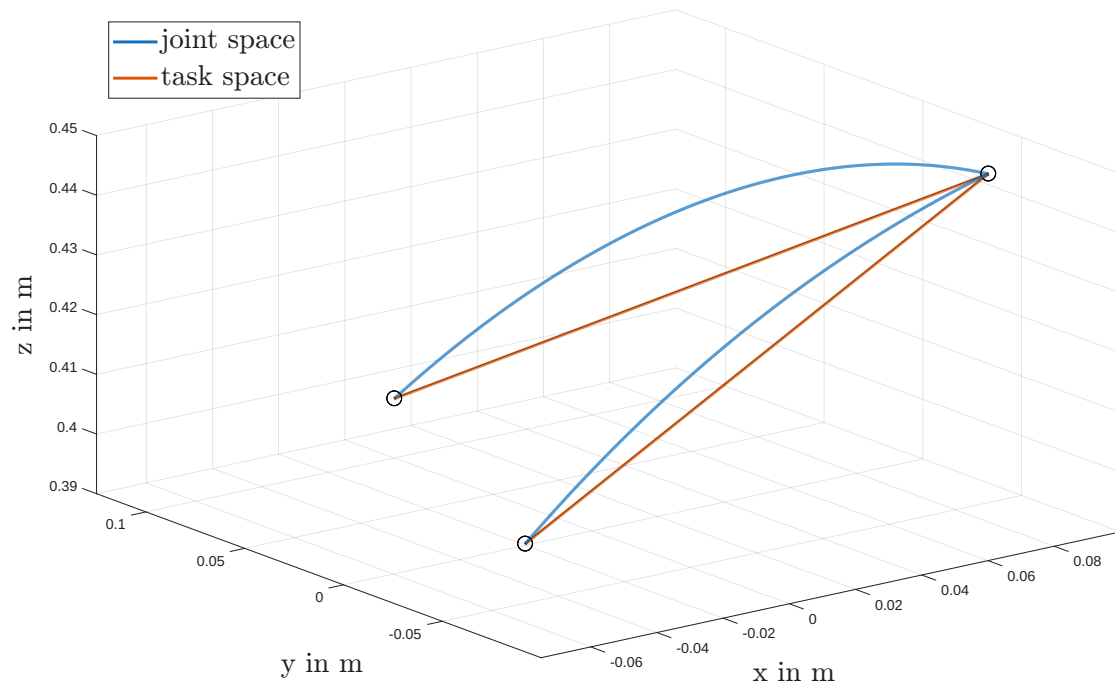


Figure 5.2: Velocity and acceleration of the joints for joint and task space motion

It is apparent that the joint velocities and accelerations in task space (right column) are much smaller than in joint space (left column). This is the cause of the motion taking significantly longer. The difference in the path that the end-effector takes in both motion modes can be viewed in figure 5.3.

**Figure 5.3:** End-effector position in joint and task space

Chapter 6

Continuous Trajectory

The next task was to implement a continuous trajectory between the chosen task space points from before (table 4.1), but without any full-stops at the via position. To achieve this, a cubic spline interpolation has been implemented. The implementation is done in the function *jspace_continuous.m*, which needs to be run before starting the simulation. The joint position and velocities can be viewed in figure 6.1.

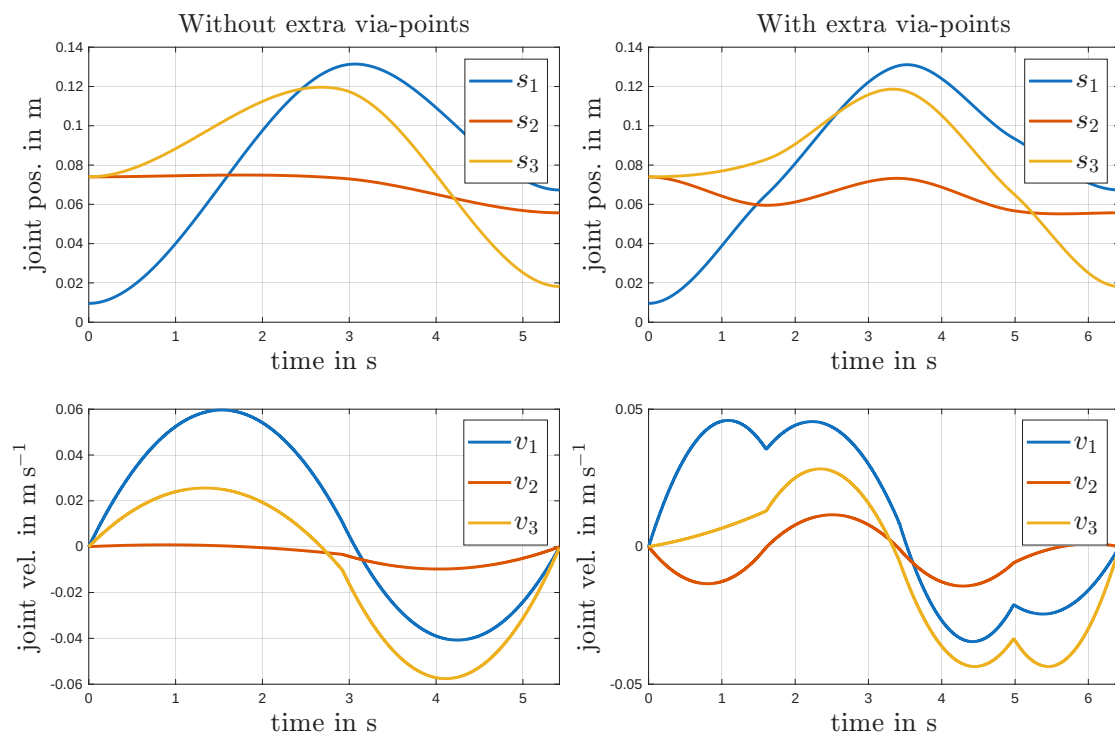


Figure 6.1: Position and velocity of joints using a continuous trajectory, with and without extra via-points

The trajectory was run twice, once with the three waypoints from before and once using two additional via-points right in the middle of waypoint 1 and 2 as well as between 2 and 3. It is apparent, that there are some abrupt changes in velocity, which is especially visible in the graphs containing

extra via-points. To view the path, which the end-effector follows, both situations are plotted in figure 6.2.

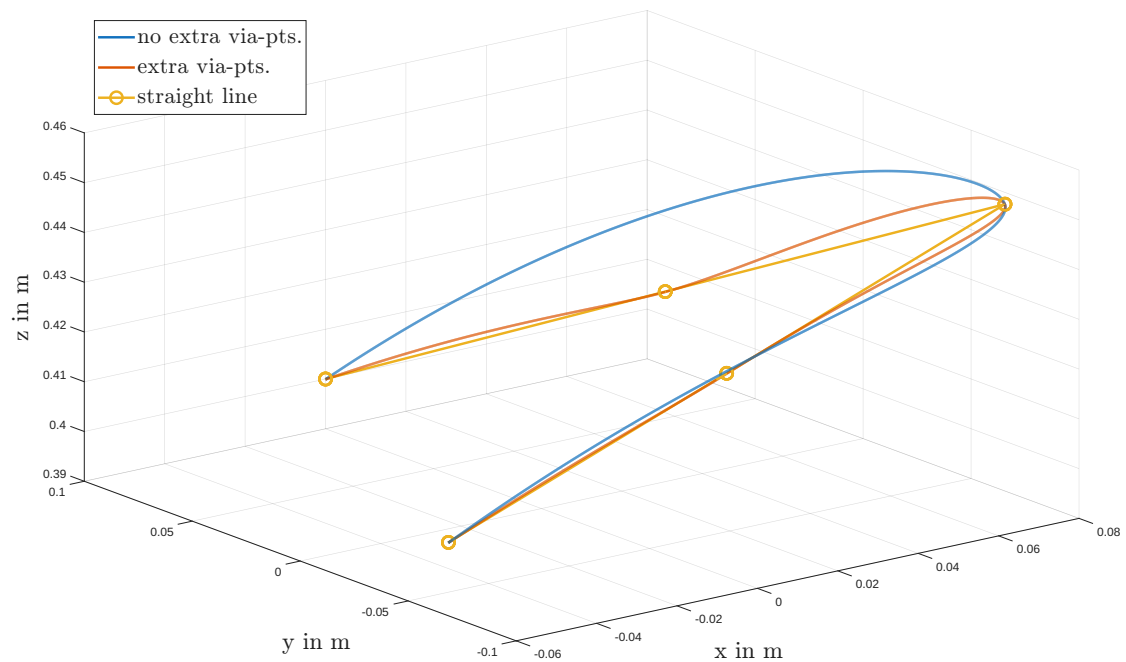


Figure 6.2: End-effector x and y position with and without extra via-points

Without via-points, the end-effector deviates from the straight line quite heavily, even more than for the trapezoidal trajectory in joint space. By choosing two extra via-points in the middle, these deviations can be reduced quite significantly, however.

How all the motion types that have been discussed in this report perform in terms of duration time and the amount they deviate from the straight line is depicted in table 6.1.

Table 6.1: Comparison of motion types in terms of duration and deviation from straight line

Motion type	duration in s	deviation in mm
joint space trapezoidal	5.425	6.48
task space trapezoidal	9.4874	0.06786
continuous w.o. via points	5.4211	12.06
continuous with. via points	6.4217	2.319

It is apparent, that the task space motion takes the largest amount of time for the movement, but follows the straight line almost perfectly. The standard joint space motion with a full stop at the via points forms a middle ground between speed and deviation from the straight line. The fastest but most inaccurate one is the continuous trajectory without any extra via points. When adding these extra points, the movement takes slightly more time, but yields the second most accurate line

following properties after the task space motion.

Bibliography

- [1] Prof. Dr. Mehmet Ismet Can Dede, *Parallel Manipulators: Kinematic Analyses of Delta 360 by IGUS*, MCI, 2021.